EE2703 - Applied Programming Lab (Report 2)

Nithin G - EE24B046

1. Objective

The objective of the assignment is to analyze the efficiency of different keyboard layouts as well as to provide the plots of the loss vs number of iterations

2. The code

The below code explains the functions that were implemented apart from those already given.

The implementations of **temperature,no_of_iterations,alpha** can tweaked at the main function on the `line-253` of the `kbd_optim.py` code.

```
def preprocess_text(text: str, chars: str) -> str:
    preprocessed_content=''
    for ch in text.lower():
        if ch not in chars:
            preprocessed_content += ' '
        else:
            preprocessed_content += ch
    return preprocessed_content
```

The text in the text file is mapped to lowercase alphabets and those not in the set of alphabets are mapped to the `' '` character

```
def path_length_cost(text: str, layout: Layout) -> float:
    path_cost = 0
    text_length = len(text)
    for i in range(text_length - 1):
        path_cost += math.dist(layout[text[i]],layout[text[i+1]])
    return path_cost
```

The cost defined as the sum of euclidean distances across consecutive characters, depending on the layout and coordinates in `qwerty_coordinates` function

```
def simulated_annealing(text: str,layout: Layout,params: SAParams,rng: random.Random,
) -> Tuple[Layout, float, List[float], List[float]]:
    ..................................................
    for i in range(number_iterations):
        #Updating the temperature after every iteration
        T=t0*(alpha**i)
        current_layout=prev_layout.copy()
        #Swap 2 random keys
```

```
        key1,key2=rng.sample(keys_layout,2) current_layout[key1],current_layout[key2]=
        current_layout[key2],current_layout[key1]
        current_cost=path_length_cost(text,current_layout)
        min_cost=min(min_cost,current_cost) #Updating the minimum cost
        if min_cost == current_cost:
            optimized_layout = current_layout
        delta=current_cost-prev_cost #Defining delta parameter
        #Simulated annealing algorithm
        if delta<0:
            prev_layout=current_layout
            prev_cost=current_cost
        else:
            probability=math.exp(-delta/T)
            if rng.random()<probability:
                prev_layout=current_layout
                prev_cost=current_cost
    ..................................................
```

A short snippet of function is shown here. The function follows the simulated annealing where 2 characters are swapped at random and the optimized layout is updated by a monte carlo way, whereas also updating the probability of an optimal descent if `random number [p]` was less than $e^{-\frac{\Delta}{T}}$, where $\Delta =$ `current_cost - prev_cost`.

3. Analysis

A python file called `Code for text` was used to generate a random series of alphabets including the space. [By default 1000 characters] and was saved in `"long_text.txt"`

The results are as follows: For Temp = 1, $\alpha = 0.99$ and 2000 iterations.

1. Baseline [QWERTY assignment] cost: 3243.1923
2. Optimized cost: 2587.2331 (improvement 655.9592)

For Temp = 10, $\alpha = 0.99$ and 2000 iterations.

1. Baseline [QWERTY assignment] cost: 3243.1923
2. Optimized cost: 2677.1914 (improvement 566.0008)

For Temp = 1, $\alpha = 0.99$ and 500 iterations.

1. Baseline [QWERTY assignment] cost: 3243.1923
2. Optimized cost: 2614.8418 (improvement 628.3505)

For Temp = 5, $\alpha = 0.99$ and 2000 iterations.

1. Baseline [QWERTY assignment] cost: 3243.1923
2. Optimized cost: 2599.7276 (improvement 643.4647)

For Temp = 1, $\alpha = 0.99$ and 100 iterations.

1. Baseline [QWERTY assignment] cost: 3243.1923
2. Optimized cost: 2744.9915 (improvement 498.2008)

Hence it can be concluded that from the above the performance of the optimizer `increase with increase in number of iterations and decreases with increase in Temperature`

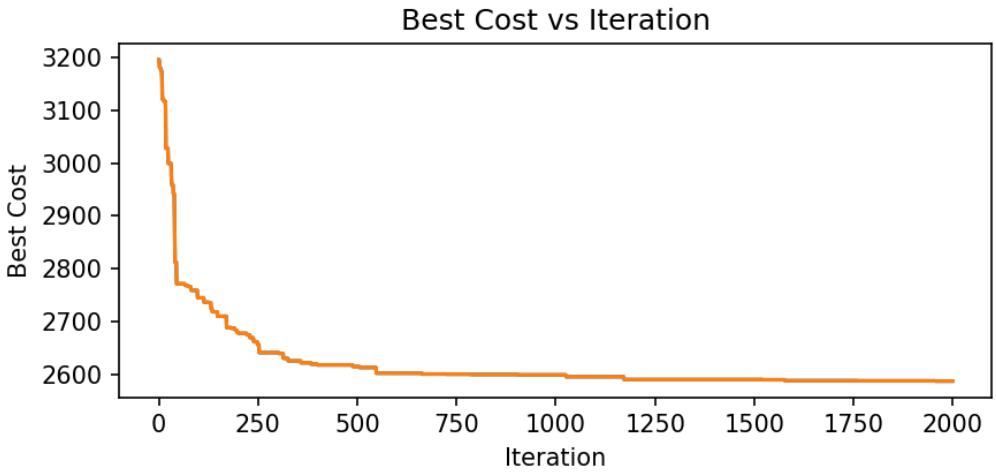

Figure 1: Cost vs Number of iterations

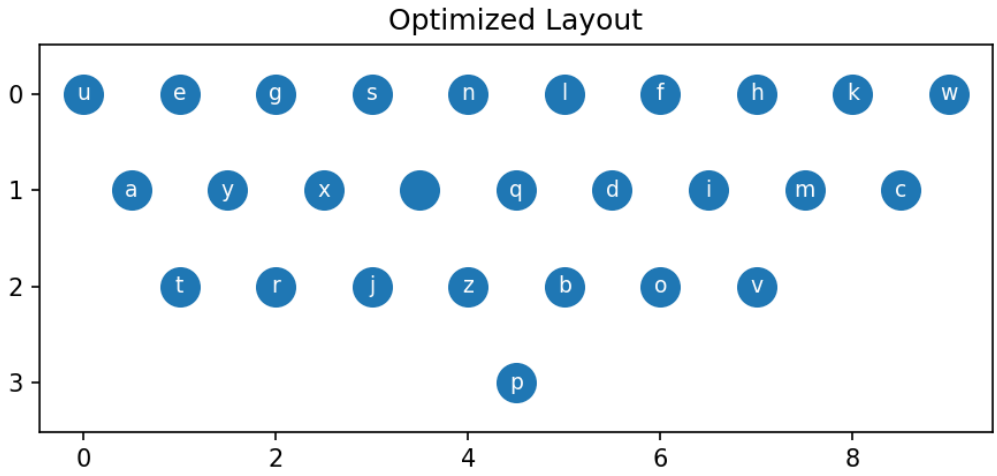

Figure 2: Optimized Layout

The above images are for:

- $T = 1$
- $\alpha = 0.99$
- Number of iterations = 2000